

CORSO NODE.JS + EXPRESS + JWT

Express base e struttura del progetto

Dispensa del corso

Giorno 1 — Parte 2

DOCENTE

Massimo Petrolati

Product Manager · Namirial SpA

Edizione 2026 · Documento di studio per studenti livello base

Indice

Cosa troverai in questa dispensa

1	Perché Express	7
1.1	Il limite di <code>http</code> puro	7
1.2	Cos'è Express	8
1.3	In che cosa NON ti aiuta	8
1.4	Alternative che sentirai nominare	8
2	Setup di un progetto Express	11
2.1	Inizializzare il progetto	11
2.2	Abilitare ES Modules	11
2.3	Installare Express e nodemon	12
2.4	Strutturare le cartelle del progetto	12
2.5	Il primo server Express	13
2.6	Configurare gli script npm	13
2.7	Il <code>.gitignore</code> che useremo sempre	14
3	Routing con Express	16
3.1	I quattro metodi base	16
3.2	Parametri di route (<code>:param</code>)	16
3.3	Query string (<code>req.query</code>)	17
3.4	Body JSON (<code>req.body</code>)	17
3.5	Rispondere: <code>res.send</code> , <code>res.json</code> , <code>res.status</code>	18
3.6	Riscriviamo in Express il server del Giorno 1	18
4	Il concetto di middleware	21
4.1	Cos'è un middleware (in una riga)	21
4.2	Middleware built-in	21
4.3	Middleware di terze parti	22
4.4	Scrivere un middleware custom	22
4.5	Middleware con argomenti (factory)	23
4.6	L'ordine di registrazione conta (parecchio)	23
5	Organizzare il codice	26
5.1	Il problema: tutto in <code>app.js</code> non scala	26
5.2	Cos'è <code>express.Router()</code>	26

5.3	La struttura a cartelle che useremo	27
5.4	Separare <code>app.js</code> e <code>server.js</code>	27
5.5	Esempio completo: refactor del server del capitolo 3	28
6	CRUD in-memory di utenti finti	31
6.1	Lo scopo dell'esercizio	31
6.2	Il "database" in memoria	31
6.3	Le route di lettura: GET	32
6.4	La route di creazione: POST	33
6.5	La route di aggiornamento: PUT	34
6.6	La route di eliminazione: DELETE	34
6.7	Mettere tutto insieme	35
6.8	Testare con Postman / REST Client	35
6.9	Esercizio extra: endpoint di ricerca	36

Introduzione

Da Node “nudo” a Express, in due ore

Questa dispensa raccoglie tutto il materiale della **seconda parte del Giorno 1** del corso Node.js + Express + JWT. È il naturale proseguimento della Parte 1 (“Fondamenti di Node.js”): assumiamo che tu sappia cosa sono il runtime, l’event loop, gli ES Modules, npm e il modulo `http` nativo. Se qualcuna di queste parole ti suona nuova, prima di proseguire torna sulla dispensa precedente, almeno per un veloce ripasso.

Da dove veniamo e dove andiamo

Alla fine della Parte 1 abbiamo costruito un piccolo server con il modulo `http` nativo. Funzionava, ma ci eravamo lasciati con una domanda: *quante righe ci sono volute per leggere un body JSON? E se domani aggiungiamo venti endpoint?*

In queste due ore rispondiamo a quella domanda. Installeremo **Express**, vedremo come riscrivere lo stesso server in tre righe invece che in trenta, e introdurremo i mattoncini fondamentali del lavoro da backend developer: **routing dichiarativo**, **middleware**, **router modulari**, separazione tra `app.js` e `server.js`. Chiuderemo costruendo insieme un piccolo CRUD di utenti finti, in memoria, che farà da scheletro al progetto Auth API delle giornate successive.

Come è strutturata

La dispensa segue i sei blocchi della lezione:

- Perché Express, cosa risolve e cosa no.
- Come si setta un progetto Express da zero, con `nodemon` e script npm.
- Routing dichiarativo: metodi, parametri, query string, body JSON.
- Concetto di middleware: built-in, di terze parti, custom, ordine di esecuzione.
- Organizzazione del codice: `express.Router()`, struttura a cartelle, separazione `app.js` / `server.js`.
- Esercizio guidato: CRUD in-memory di utenti finti, testabile con Postman.

I box che incontrerai

- **CODICE:** riferimento al mini-progetto pronto nella cartella `codice-esempi/giorno-1-parte-2/`. Apri il file, esegui, modificalo.
- **Suggerimento:** consiglio pratico per evitare problemi tipici.
- **Attenzione:** trappola comune o cosa NON fare.

Come imparare davvero

La cosa più importante che farai oggi è **cancellare e riscrivere**. Lascia che Express ti sembri “magia” per i primi cinque minuti, poi fermati e prova a riscrivere a memoria il primo server. La seconda volta lo capirai davvero. Vale lo stesso per il CRUD: dopo averlo seguito in aula, chiudi il file e prova a rifarlo da zero. È così che si fissa.

Dove trovi il codice

Tutti gli esempi della dispensa sono pronti nel **repository GitHub del corso**:

github.com/mpetrolati/2026-corso-backend-nodejs-express-its

Clonalo **una volta sola** all’inizio del corso e usalo come riferimento per ogni esempio:

```
$ git clone https://github.com/mpetrolati/2026-corso-backend-nodejs-express-its.git
$ cd 2026-corso-backend-nodejs-express-its
```

I file di questa dispensa stanno nella cartella `codice-esempi/giorno-1-parte-2/`.

CAPITOLO 01

Perché Express

Cosa risolve, cosa non risolve, come si colloca

1. Perché Express

1.1 Il limite di http puro

Nella Parte 1 abbiamo scritto il nostro primo server con il modulo `http` di Node. Lo riprendiamo qui per fissare il punto di partenza:

```
import http from 'http';

const server = http.createServer((req, res) => {
  if (req.url === '/' && req.method === 'GET') {
    res.end('Home');
  } else if (req.url === '/about' && req.method === 'GET') {
    res.end('About');
  } else {
    res.statusCode = 404;
    res.end('Not Found');
  }
});

server.listen(3000);
```

Funziona, ma già con due endpoint il codice puzza. Aggiungiamo l'idea di leggere un body JSON in POST e diventa così:

```
if (req.method === 'POST' && req.url === '/echo') {
  let body = '';
  req.on('data', chunk => { body += chunk.toString(); });
  req.on('end', () => {
    try {
      const dati = JSON.parse(body);
      res.setHeader('Content-Type', 'application/json');
      res.end(JSON.stringify({ ricevuto: dati }));
    } catch {
      res.statusCode = 400;
      res.end('JSON non valido');
    }
  });
}
```

Per un endpoint solo, dieci righe. Per venti endpoint, una catena infinita di `if/else if/else`, parsing del body ripetuto, header da settare a mano ogni volta, gestione errori sparsa. Non è

sostenibile.

1.2 Cos'è Express

Express è un piccolo framework che si mette sopra al modulo `http` nativo e copre con un'API più ergonomica i 90% dei pattern ricorrenti. Non sostituisce `http`: lo usa sotto al cofano. Quando scrivi `app.get(...)` o `app.post(...)`, Express alla fine chiama `http.createServer` per te.

Quello che Express ti regala:

- **Routing dichiarativo:** scrivi `app.get('/users')` invece di `if (req.url === '/users' && req.method === 'GET')`.
- **Middleware:** piccoli pezzi di codice che intercettano la richiesta in modo componibile (logging, parsing del body, autenticazione, CORS, gestione errori). Su questo torneremo nel capitolo 4.
- **Una community enorme:** per ogni problema che incontrerai esiste già un middleware npm pronto. Validazione input? Esiste. Rate limiting? Esiste. Cookie sicuri? Esiste.

Numeri per inquadrarlo

Express è nato nel 2010, ha superato i **30 milioni di download a settimana** su npm, ed è il framework di default per le API REST in Node. È sotto al cofano di startup, banche, e anche di tool che usi tutti i giorni. È il “Spring Boot di Node”, se la metafora ti aiuta.

1.3 In che cosa NON ti aiuta

Express è volutamente **minimale**. Quello che ti dà è poco e ben fatto: routing, middleware, e un sottile wrapper attorno a `req/res`. Quello che non ti dà:

- Nessun ORM o adapter di database (devi sceglierti tu `mysql2`, `pg`, `mongoose`, `sqlite3` + `sqlite`, eccetera).
- Nessuna autenticazione integrata (anche qui scegli tu: JWT, sessioni, OAuth via Passport.js).
- Nessuna struttura di progetto consigliata. Il file principale lo organizzi come vuoi, ed è una libertà a doppio taglio: senza disciplina si finisce con `app.js` da 2000 righe.

Questa filosofia “poche cose, fatte bene” è il motivo per cui Express è ancora rilevante dopo 16 anni: ogni progetto ci costruisce sopra il pezzo che gli serve.

1.4 Alternative che sentirai nominare

Express non è solo: il mondo Node ha un piccolo ecosistema di framework concorrenti. Te li elenco rapidamente, così quando li senti in giro sai inquadrarli, ma nel corso useremo **Express 4.x** perché è quello che troverai in azienda il 90% delle volte.

- **Fastify**: più giovane, più veloce (benchmark migliori), API simile a Express con qualche novità. Sta crescendo.
- **Koa**: scritto dagli stessi autori di Express, più minimale, usa massicciamente `async/await` e i “context” invece di `req/res`. Curva di apprendimento un filo più ripida.
- **NestJS**: non è un concorrente diretto, è un framework “opinionato” costruito sopra Express (o Fastify), con un’architettura simile ad Angular o Spring (decoratori, dependency injection, moduli). Adatto a team grandi e progetti enterprise.
- **Hono**: nuovissimo, pensato per gli edge runtime (Cloudflare Workers, Deno, Bun). Da tenere d’occhio.

Quale impararne prima?

Express, senza dubbi. È il più diffuso, il più documentato, il più presente nei colloqui di lavoro. Una volta che lo padroneggi, passare a Fastify o Koa richiede mezza giornata: i concetti (routing, middleware) sono gli stessi.

CAPITOLO 02

Setup di un progetto Express

Da `npm init` al primo server con `nodemon`

2. Setup di un progetto Express

2.1 Inizializzare il progetto

Apri un terminale, crea una cartella vuota e inizializza npm:

```
$ mkdir mio-primo-express
$ cd mio-primo-express
$ npm init -y
```

L'opzione `-y` accetta tutti i valori di default. Se vuoi compilare i campi a mano (nome del progetto, autore, licenza), togli il `-y` e rispondi alle domande. Quello che ottieni è un `package.json` simile a questo:

```
{
  "name": "mio-primo-express",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

2.2 Abilitare ES Modules

Lo abbiamo già visto nella Parte 1, ma è il primo errore che farai dimenticare questo passaggio. Apri il `package.json` e aggiungi una riga:

```
{
  "name": "mio-primo-express",
  "version": "1.0.0",
  "type": "module",
  "main": "src/server.js",
  ...
}
```

Senza `"type": "module"`, Node tratterà tutti i `.js` come CommonJS e quando proverai `import express from 'express'` ti darà l'errore “Cannot use import statement outside a module”. Per tutto il corso questa riga deve esserci.

2.3 Installare Express e nodemon

Express è una dipendenza “vera” (serve sia in sviluppo che in produzione). `nodemon` è invece un tool di sviluppo: riavvia il server automaticamente quando salvi un file. Si installano così:

```
# dipendenza di produzione: finisce in "dependencies"
$ npm install express

# dipendenza di sviluppo: finisce in "devDependencies"
$ npm install --save-dev nodemon
```

Dopo l'installazione, dai un'occhiata al `package.json`: vedrai apparire le sezioni `dependencies` e `devDependencies`. È la prima volta che lo guardi seriamente: questo file è il **manifest del tuo progetto**, e chiunque cloni il repo poi fa `npm install` per ricostruire la stessa configurazione.

Suggerimento

Le versioni che vedi (es. `^4.19.2`) usano il prefisso `^` che significa “compatibile minor / patch”. Per i corsi e i tutorial va bene. In produzione si tende a essere più rigorosi (`package-lock.json` viene committato proprio per questo).

2.4 Strutturare le cartelle del progetto

Express non impone una struttura, ma noi la imponiamo a noi stessi fin da subito. È un'abitudine sana e ci porterà dritti al progetto finale del giorno 3.

```
mio-primo-express/
|-- src/
|   |-- app.js          <-- configurazione Express (middleware, route)
|   `-- server.js       <-- entry point (avvia il listener)
|-- .gitignore
|-- package.json
`-- package-lock.json
```

La separazione tra `app.js` e `server.js` sembra eccessiva per ora, ma ha una motivazione precisa che vedremo nel capitolo 5: ti permette di importare `app` nei test senza far partire davvero il server.

2.5 Il primo server Express

Crea il file `src/app.js` con questo contenuto minimo:

```
import express from 'express';

const app = express();

app.get('/', (req, res) => {
  res.send('Hello Express!');
});

export default app;
```

E il file `src/server.js` con questo:

```
import app from './app.js';

const PORT = 3000;

app.listen(PORT, () => {
  console.log(`Server in ascolto su http://localhost:${PORT}`);
});
```

2.6 Configurare gli script npm

Apri di nuovo `package.json` e modifica la sezione `scripts`:

```
"scripts": {
  "start": "node src/server.js",
  "dev": "nodemon src/server.js"
}
```

Ora puoi lanciare il server in due modi:

```
$ npm start      # produzione: si avvia, basta
$ npm run dev    # sviluppo: riparte automaticamente a ogni salvataggio
```

Apri il browser su `http://localhost:3000` e dovresti vedere “Hello Express!”.

Errore tipico

Se vedi `EADDRINUSE: port 3000 already in use`, c'è già un altro processo che usa la porta 3000 (magari un `node` rimasto aperto da prima). Chiudilo con `Ctrl+C` nella finestra giusta, oppure cambia la porta nel `server.js`.

2.7 Il `.gitignore` che useremo sempre

Prima di committare nulla, crea il file `.gitignore` alla radice del progetto con queste righe:

```
node_modules/  
.env  
.env.local  
*.log  
.DS_Store
```

è la regola d'oro: questa cartella può contenere centinaia di MB di codice di terze parti, e si ricostruisce da `package.json` con `npm install`. **Non si committa mai.** `.env` lo vedremo nel Giorno 2 quando parleremo di segreti, ma è bene predisporlo da subito.

CODICE: `giorno-1-parte-2/01-express-hello/`

CAPITOLO 03

Routing con Express

Metodi, parametri, query string, body JSON

3. Routing con Express

3.1 I quattro metodi base

Express espone un metodo per ogni verbo HTTP. La sintassi è sempre la stessa: il primo argomento è il path, il secondo è la callback che riceve `req` (richiesta) e `res` (risposta).

```
app.get('/users', (req, res) => res.send('Lista utenti'));
app.post('/users', (req, res) => res.send('Crea utente'));
app.put('/users/:id', (req, res) => res.send('Aggiorna utente'));
app.delete('/users/:id', (req, res) => res.send('Elimina utente'));
```

I metodi esistono per tutti i verbi HTTP: `patch`, `options`, `head`, eccetera. Nella pratica useremo soprattutto `GET`, `POST`, `PUT`, `DELETE` e occasionalmente `PATCH`.

Convenzione REST

Nelle API REST si associa il verbo all'operazione: `GET` per leggere, `POST` per creare, `PUT/PATCH` per aggiornare, `DELETE` per cancellare. Non è una regola della tecnologia, è una convenzione condivisa: rispettarla rende le API "leggibili" a chiunque ne abbia viste qualche altra.

3.2 Parametri di route (:param)

Quando una parte del path è dinamica (es. l'id dell'utente), si usa il prefisso `:`.

```
app.get('/users/:id', (req, res) => {
  const id = req.params.id;
  res.send(`Hai chiesto l'utente ${id}`);
});
```

Puoi metterne più di uno:

```
app.get('/users/:userId/posts/:postId', (req, res) => {
  const { userId, postId } = req.params;
  res.send(`Utente ${userId}, post ${postId}`);
});
```


Attenzione

I parametri di route arrivano sempre come **stringhe**, anche se sembrano numeri. Se ti serve un `Number`, convertilo esplicitamente: `Number(req.params.id)` o `parseInt(req.params.id, 10)`. Stesso discorso per query string e body JSON che arriva in forma non tipizzata.

3.3 Query string (`req.query`)

Quando la richiesta è del tipo `/users?nome=Mario&eta=30`, Express ti popola `req.query` con un oggetto:

```
app.get('/users', (req, res) => {
  const { nome, eta } = req.query;
  res.send(`Cerco utenti chiamati ${nome ?? 'qualunque'}, eta ${eta ?? '?'}`);
});
```

In Node puro avremmo dovuto usare `new URL(...)` e `searchParams.get(...)`. Con Express, è gratis.

3.4 Body JSON (`req.body`)

Per leggere un body JSON in una POST o PUT, serve attivare il middleware `express.json()`. È una riga che metti una volta sola in cima all'app:

```
import express from 'express';
const app = express();

app.use(express.json()); // <-- abilita il parsing del body JSON

app.post('/echo', (req, res) => {
  res.json({ ricevuto: req.body });
});
```

Da ora in poi, ogni richiesta con header `Content-Type: application/json` avrà il body pre-parsato in `req.body`. Niente più `req.on('data', ...)` a mano.

Confronto col modulo http nativo: per leggere lo stesso body in Node puro servivano una decina di righe (accumulo dei chunk, `try/catch` sul `JSON.parse`, status code per il caso d'errore). Con Express si scrive `app.use(express.json())` una volta e basta. È esattamente la promessa che il capitolo 1 ti faceva: *Express comprime tutto il boilerplate ripetitivo*.

3.5 Rispondere: `res.send`, `res.json`, `res.status`

La risposta si costruisce su `res`. I metodi che userai il 95% delle volte:

- `res.send(qualcosa)`: invia una risposta generica. Se passi una stringa, manda `text/plain`. Se passi un oggetto, lo serializza in JSON automaticamente (ma è meglio usare `res.json` esplicitamente).
- `res.json(oggetto)`: stesso effetto di `res.send(oggetto)` ma più chiaro: chi legge il codice capisce subito che stiamo facendo un'API JSON.
- `res.status(codice)`: imposta lo status code. Si concatena: `res.status(201).json({...})`.
- `res.sendStatus(codice)`: scorciatoia per "rispondi solo con uno status code, niente body".

```
app.post('/users', (req, res) => {
  const nuovoUtente = { id: 1, ...req.body };
  // 201 = Created, lo standard per la creazione di una risorsa
  res.status(201).json(nuovoUtente);
});

app.get('/healthz', (req, res) => {
  res.sendStatus(200); // 200 OK senza body
});
```

3.6 Riscriviamo in Express il server del Giorno 1

Ricordi il server del Giorno 1, quello con `http` puro? Lo riscriviamo da capo in Express, una riga alla volta:

```
import express from 'express';

const app = express();
app.use(express.json());

app.get('/', (req, res) => res.send('Home'));
app.get('/about', (req, res) => res.send('About'));
app.get('/saluta', (req, res) => {
  const nome = req.query.nome ?? 'sconosciuto';
  res.send(`Ciao ${nome}!`);
});
app.post('/echo', (req, res) => {
  res.json({ ricevuto: req.body });
});

app.listen(3000, () => console.log('Server su http://localhost:3000'));
```

Confronta queste righe con la versione del Giorno 1: lo stesso identico server, passato da una settantina di righe a una quindicina. **Questo è il valore di Express in una slide.**

CODICE: [giorno-1-parte-2/02-express-routing/](#)

CAPITOLO 04

Il concetto di middleware

Built-in, di terze parti, custom, ordine di esecuzione

4. Il concetto di middleware

4.1 Cos'è un middleware (in una riga)

Un middleware è una funzione che riceve `(req, res, next)`, fa qualcosa, e poi chiama `next()` per passare il controllo al successivo.

Detto in altri termini: Express vede una richiesta in arrivo come un treno che attraversa una serie di stazioni. Ogni stazione è un middleware: può guardare il treno, decorarlo, fermarlo, o lasciarlo passare. La sequenza di stazioni la decidi tu con `app.use(...)`.

```
function logger(req, res, next) {  
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);  
  next(); // <-- passa la richiesta al prossimo middleware  
}  
  
app.use(logger);
```

Quel `next()` è la chiave: senza, la richiesta resta “bloccata” nel middleware e il client non riceve mai risposta.

Cosa può fare un middleware

Decorare `req` (es. aggiungere `req.user` dopo aver verificato un token), modificare `res` (es. settare header CORS), oppure **interrompere la pipeline** rispondendo subito (es. “token mancante → 401, niente next”). Sono gli stessi tre poteri che useremo per il middleware di autenticazione JWT nel Giorno 3.

4.2 Middleware built-in

Express ne porta con sé due fondamentali, già visti nel capitolo 3:

- `express.json()`: parser del body JSON. Popola `req.body`.
- `express.urlencoded({ extended: true })`: parser del body in formato form-urlencoded (quello che mandano i form HTML classici).

```
app.use(express.json());  
app.use(express.urlencoded({ extended: true }));
```

Esisteva anche `express.static` per servire file statici (HTML, immagini, CSS), ma nel corso non

ci serve perché stiamo costruendo solo l'API.

4.3 Middleware di terze parti

Quando ti serve qualcosa di standard, prima di scriverlo a mano controlla npm: nove volte su dieci esiste un middleware pronto. Quelli che incontrerai più spesso:

- `morgan`: logger formattato per le richieste HTTP. Bellissimo l'output colorato in dev.
- `cors`: gestisce gli header CORS quando un frontend (su un'altra origine) chiama la tua API.
- `helmet`: setta un pacchetto di header di sicurezza con una riga.
- `express-rate-limit`: protezione anti brute-force, utile prima di endpoint di login.

Esempio veloce di `morgan`:

```
$ npm install morgan
```

```
import morgan from 'morgan';  
app.use(morgan('dev')); // log colorato e conciso, perfetto in sviluppo
```

Da ora in poi, ogni richiesta verra loggata nel terminale con metodo, path, status code e tempo di risposta. Comodissimo.

4.4 Scrivere un middleware custom

Scriviamone uno noi. Obiettivo: misurare quanto tempo impiega ogni richiesta a essere processata.

```
function durata(req, res, next) {  
  const inizio = Date.now();  
  res.on('finish', () => {  
    const ms = Date.now() - inizio;  
    console.log(`${req.method} ${req.url} -> ${res.statusCode} (${ms}ms)`);  
  });  
  next();  
}  
  
app.use(durata);
```

Cose interessanti di questo middleware:

- Salviamo il timestamp di inizio nello scope della funzione.

- Ci agganciamo all'evento `finish` di `res`, che scatta quando la risposta è stata mandata al client (qualunque sia stato il path nella pipeline).
- Chiamiamo `next()` subito, perché non vogliamo bloccare la pipeline: vogliamo solo osservarla.

Ed ecco l'output che vedrai sul terminale:

```
GET /users -> 200 (3ms)
POST /users -> 201 (18ms)
GET /users/42 -> 404 (1ms)
```

4.5 Middleware con argomenti (factory)

Quando un middleware ha bisogno di configurazione, lo si scrive come **factory**: una funzione che ritorna il middleware vero e proprio.

```
function richiediHeader(nome) {
  return function (req, res, next) {
    if (!req.headers[nome.toLowerCase()]) {
      return res.status(400).json({ errore: `Manca header ${nome}` });
    }
    next();
  };
}

app.use('/api', richiediHeader('X-Client-Id'));
```

È il pattern che useremo nel Giorno 3 per il middleware `verifyToken(req, res, next)`.

4.6 L'ordine di registrazione conta (parecchio)

Express esegue i middleware **nell'ordine in cui sono registrati**. Sembra ovvio, ma è la causa numero uno di “ma perché non funziona?” quando si comincia.

```
app.use(express.json());           // 1) parser body
app.use(logger);                   // 2) logger
app.use('/api', autentica);        // 3) verifica token su /api/*
app.get('/api/users', ...);        // 4) handler
```

Se inverti l'ordine e metti `app.use(express.json())` **dopo** la route, in quella route `req.body` sarà `undefined` e ti chiederai per mezza giornata perché. Regola pratica:

- I middleware di **parsing** (`json`, `urlencoded`, `cors`, `helmet`) vanno in **cima**.
- I middleware di **autenticazione** vanno **prima delle route che proteggono**.
- Il middleware di **gestione errori** (firma a 4 parametri) va in **coda**, dopo tutte le route. Lo vedremo nel Giorno 2.

Anteprima Giorno 2

Il middleware di gestione errori ha una firma diversa: `(err, req, res, next)`, quattro parametri invece di tre. Express lo riconosce dalla firma e lo invoca solo quando qualche middleware precedente fa `next(err)` passandogli un errore. Approfondiamo nel Giorno 2, ma intanto tienilo a mente.

CODICE: `giorno-1-parte-2/03-express-middleware/`

CAPITOLO 05

Organizzare il codice

`express.Router()`, cartelle, separazione `app.js` / `server.js`

5. Organizzare il codice

5.1 Il problema: tutto in `app.js` non scala

Per ora abbiamo messo tutte le route nel file principale. Va bene con cinque endpoint, è insopportabile con cinquanta. Diventa subito illeggibile, è difficile cercare cosa fa una route, e ogni piccola modifica rischia di toccare un file enorme.

La soluzione che Express ci offre è semplice: dividere le route per **area logica**, ognuna in un file dedicato, agganciandole a un base path comune. Lo strumento che usiamo si chiama `express.Router()`.

5.2 Cos'è `express.Router()`

`express.Router()` crea un “mini-Express” a cui registrare route e middleware. Lo puoi pensare come una scatola di endpoint che poi monti dentro l'app principale.

```
// src/routes/userRoutes.js
import { Router } from 'express';

const router = Router();

router.get('/', (req, res) => res.send('Lista utenti'));
router.get('/:id', (req, res) => res.send(`Utente ${req.params.id}`));
router.post('/', (req, res) => res.status(201).send('Creato'));

export default router;
```

E poi nel file principale:

```
// src/app.js
import express from 'express';
import userRoutes from './routes/userRoutes.js';

const app = express();
app.use(express.json());

app.use('/api/users', userRoutes); // <-- monta il router sul path

export default app;
```

Cosa è successo: dentro `userRoutes.js` i path sono `/`, `/:id`, `/` ma quando montati con `app.use('/api/users', ...`

diventano `/api/users`, `/api/users/:id`, `/api/users`. Express concatena automaticamente.

Perché è così potente

Aggiungere una nuova area dell'API ora significa creare un nuovo file dentro `routes/`, scrivere il router, e montarlo in `app.js` con una riga. Niente `if/else` sparsi in giro, niente conflitti tra route "vicine" di aree diverse. È la prima cosa che ti aspetti che facciano in un colloquio di lavoro come backend developer junior.

5.3 La struttura a cartelle che useremo

Da qui in poi, e fino al progetto finale del Giorno 3, organizzeremo i progetti così:

```
src/
|-- routes/      <-- file con i router (uno per area: users, auth, posts...)
|-- controllers/ <-- funzioni handler che leggono req e scrivono res
|-- middlewares/ <-- middleware custom (logger, auth, error handler)
|-- app.js       <-- configurazione Express (middleware + montaggio router)
`-- server.js    <-- entry point: avvia il listener
```

Per ora introduciamo solo `routes/` e la separazione `app.js/server.js`. Nel Giorno 2 aggiungeremo `controllers/`, `services/` e `repositories/`. Nel Giorno 3 `middlewares/` per il middleware JWT e `utils/` per gli helper.

5.4 Separare `app.js` e `server.js`

Hai notato che lo facciamo dal capitolo 2. È il momento di spiegare il perché.

- `app.js` **costruisce** l'app Express: registra middleware e router, ed esporta l'istanza pronta. Non chiama mai `listen`.
- `server.js` **usa** l'app: la importa e fa partire il listener sulla porta.

Vantaggi concreti:

- Quando arriveranno i test (`jest` + `supertest`, fuori scope del corso), potrai importare direttamente `app` e fare richieste finte senza far girare un server vero. Veloce e pulito.
- Se domani ti serve servire la stessa app via WebSocket o HTTPS, modifichi solo `server.js`: `app.js` resta intatto.
- È una buona abitudine: nei progetti seri lo trovi sempre fatto così, anche se per ora ti sembra un dettaglio.

5.5 Esempio completo: refactor del server del capitolo 3

Riprendiamo il server del capitolo 3 (Home, About, Saluta, Echo) e mettiamolo in forma. Iniziamo separando le route home da quelle “utility”.

```
// src/routes/homeRoutes.js
import { Router } from 'express';
const router = Router();

router.get('/', (req, res) => res.send('Home'));
router.get('/about', (req, res) => res.send('About'));

export default router;
```

```
// src/routes/utilityRoutes.js
import { Router } from 'express';
const router = Router();

router.get('/saluta', (req, res) => {
  const nome = req.query.nome ?? 'sconosciuto';
  res.send(`Ciao ${nome}!`);
});

router.post('/echo', (req, res) => {
  res.json({ ricevuto: req.body });
});

export default router;
```

```
// src/app.js
import express from 'express';
import homeRoutes from './routes/homeRoutes.js';
import utilityRoutes from './routes/utilityRoutes.js';

const app = express();
app.use(express.json());

app.use('/', homeRoutes);
app.use('/util', utilityRoutes);

export default app;
```

```
// src/server.js
import app from './app.js';

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server su http://localhost:${PORT}`);
});
```

Quattro file dove prima ce n'era uno. Sembra più lavoro, ma è la struttura che useremo per tutto il corso e che ti porterà al progetto Auth API senza dover riscrivere niente.

CODICE: `giorno-1-parte-2/04-express-router/`

CAPITOLO 06

CRUD in-memory di utenti finti

Esercizio guidato per fissare tutto quello che abbiamo visto

6. CRUD in-memory di utenti finti

6.1 Lo scopo dell'esercizio

Abbiamo introdotto routing, middleware e router modulari. Ora li usiamo tutti insieme per costruire un piccolo **CRUD di utenti**, con tanto di `POST`, `GET`, `PUT`, `DELETE`. Nessun database vero: ci accontentiamo di un array JavaScript in memoria. Sarà lo scheletro a cui, nel Giorno 2, sostituiremo SQLite.

Endpoint che faremo girare entro fine giornata:

- `GET /api/users` lista utenti
- `GET /api/users/:id` singolo utente
- `POST /api/users` crea utente
- `PUT /api/users/:id` aggiorna utente
- `DELETE /api/users/:id` elimina utente

6.2 Il ``database" in memoria

Per simulare la persistenza ci basta un array, un contatore per gli id, e qualche funzione di lookup. Lo isoliamo in un file dedicato così domani sostituirlo con SQLite costerà cinque minuti.

```
// src/data/userStore.js
let users = [
  { id: 1, name: 'Mario Rossi', email: 'mario@example.com' },
  { id: 2, name: 'Lucia Bianchi', email: 'lucia@example.com' }
];
let nextId = 3;

export function findAll() { return users; }

export function findById(id) {
  return users.find(u => u.id === Number(id));
}

export function create({ name, email }) {
  const user = { id: nextId++, name, email };
  users.push(user);
  return user;
}

export function update(id, data) {
  const user = findById(id);
  if (!user) return null;
  Object.assign(user, data);
  return user;
}

export function remove(id) {
  const idx = users.findIndex(u => u.id === Number(id));
  if (idx === -1) return false;
  users.splice(idx, 1);
  return true;
}
```

Perché un file dedicato?

Stiamo già anticipando il **repository pattern** che vedremo nel Giorno 2. Tutta la conoscenza di “come sono fatti i dati” resta qui dentro. Le route non lo sapranno mai: chiameranno `findAll()`, `create(...)` eccetera, ignare del fatto che sotto c'è un array, un file SQLite, o Postgres.

6.3 Le route di lettura: GET

Le due route più semplici:


```
// src/routes/userRoutes.js
import { Router } from 'express';
import * as store from '../data/userStore.js';

const router = Router();

router.get('/', (req, res) => {
  res.json(store.findAll());
});

router.get('/:id', (req, res) => {
  const user = store.findById(req.params.id);
  if (!user) return res.status(404).json({ errore: 'Utente non trovato' });
  res.json(user);
});

export default router;
```

Da notare:

- Usiamo `import * as store` per importare tutte le funzioni esportate dal modulo come un namespace. È più leggibile quando le funzioni sono molte.
- `return res.status(404).json(...)` ferma il flusso dell'handler: il `return` è una buona abitudine per evitare di rispondere due volte alla stessa richiesta (errore comune e fastidioso da debuggare).
- Lo status code `404` è la convenzione REST per “risorsa non trovata”. Se mandassimo `200` con un body vuoto, il client non saprebbe distinguere “utente assente” da “errore di rete”.

6.4 La route di creazione: POST

Qui usiamo `req.body`, che funziona grazie al `express.json()` registrato in `app.js`.

```
router.post('/', (req, res) => {
  const { name, email } = req.body;

  // validazione minimale (la vera la faremo nel Giorno 2)
  if (!name || !email) {
    return res.status(400).json({
      errore: 'I campi name e email sono obbligatori'
    });
  }

  const user = store.create({ name, email });
  res.status(201).json(user);
});
```

Status code `400` per input invalido, `201` per creazione avvenuta: sono le convenzioni REST. Le useremo identiche nel progetto finale.

6.5 La route di aggiornamento: PUT

```
router.put('/:id', (req, res) => {
  const user = store.update(req.params.id, req.body);
  if (!user) return res.status(404).json({ errore: 'Utente non trovato' });
  res.json(user);
});
```

Discutibile la differenza tra `PUT` (sostituzione completa) e `PATCH` (modifica parziale): nella pratica, per piccoli CRUD si usa quasi sempre `PUT` e ci si fida del client. Quando ti servirà più rigore, esisterà `PATCH`.

6.6 La route di eliminazione: DELETE

```
router.delete('/:id', (req, res) => {
  const ok = store.remove(req.params.id);
  if (!ok) return res.status(404).json({ errore: 'Utente non trovato' });
  res.sendStatus(204); // 204 = No Content
});
```

`204 No Content` è lo status code corretto per “operazione riuscita, non ho nulla da dirti”. Eviti di mandare un body inutile.

6.7 Mettere tutto insieme

Il file `src/app.js` di questo esercizio diventa:

```
import express from 'express';
import morgan from 'morgan';
import userRoutes from './routes/userRoutes.js';

const app = express();
app.use(express.json());
app.use(morgan('dev'));

app.use('/api/users', userRoutes);

app.use((req, res) => {
  res.status(404).json({ errore: 'Route non trovata' });
});

export default app;
```

Notare l'ultimo `app.use(...)`: è il **catch-all 404**. Express scorre i middleware nell'ordine, e se nessuna route ha risposto arriva qui. Lo metti sempre per ultimo.

6.8 Testare con Postman / REST Client

Apri Postman (o l'estensione *REST Client* di VS Code, più leggera) e prova questa sequenza:

```
1. GET    http://localhost:3000/api/users
2. POST   http://localhost:3000/api/users
   Body (JSON):
   { "name": "Anna Verdi", "email": "anna@example.com" }
3. GET    http://localhost:3000/api/users
4. PUT    http://localhost:3000/api/users/3
   Body (JSON):
   { "name": "Anna Verdi Junior" }
5. DELETE http://localhost:3000/api/users/3
6. GET    http://localhost:3000/api/users
```

Vedrai l'utente apparire e sparire dalla lista. Il log di `morgan` nel terminale ti accompagna passo passo. È il momento “ah, funziona davvero” che dà soddisfazione: ricordatelo, è il motore principale della motivazione.

Limite voluto

Se fermi e riavvii il server, l'array torna al contenuto iniziale: i dati vivono solo finché vive il processo Node. Quello che vogliamo per ora: dal Giorno 2 ci servirà un vero database (SQLite).

6.9 Esercizio extra: endpoint di ricerca

Se hai tempo (o come “compito” a casa), aggiungi una route che permette di cercare utenti per nome:

```
// /api/users/search?nome=Mario
router.get('/search', (req, res) => {
  const nome = (req.query.nome ?? '').toLowerCase();
  const risultati = store.findAll().filter(u =>
    u.name.toLowerCase().includes(nome)
  );
  res.json(risultati);
});
```

Trabocchetto

Questa route va dichiarata **prima** di `router.get('/:id', ...)`. Express valuta le route nell'ordine: se ne hai una con `id` prima, una richiesta a `/search` verrebbe intercettata da `id` e trattata come “cerco l'utente con id 'search'” (con risposta 404). Buona occasione per fissare la regola dell'ordine.

CODICE: `giorno-1-parte-2/05-express-crud-inmem/`

Appendice

Cosa porti a casa e dove andiamo domani

Cosa ti porti a casa da questa lezione

- Cos'è **Express** e perché esiste: un piccolo framework sopra `http` che ti regala routing dichiarativo, middleware e ergonomia generale.
- Come si fa il setup di un progetto Express con ES Modules, `nodemon` e gli script npm `start` / `dev`.
- I quattro metodi base (`get`, `post`, `put`, `delete`), come si leggono **parametri di route**, **query string** e **body JSON**, e quali status code usare per le risposte.
- Il concetto di **middleware**: che cos'è, come scriverne uno, e perché *l'ordine di registrazione conta*.
- Come organizzare le route con `express.Router()`, perché si separano `app.js` e `server.js`, e come predisporre la struttura a cartelle che useremo per tutto il corso.
- Un **CRUD funzionante** in-memory di utenti, testabile con Postman, con i giusti status code e una bozza di repository pattern.

Cartella codice di riferimento

Tutti gli esempi sono pronti nella cartella `codice-esempi/giorno-1-parte-2/` del repository GitHub del corso:

github.com/mpetrolati/2026-corso-backend-nodejs-express-its

Se non l'hai ancora fatto, clonalo (o scaricalo come ZIP da GitHub):

```
$ git clone https://github.com/mpetrolati/2026-corso-backend-nodejs-express-its.git
$ cd 2026-corso-backend-nodejs-express-its/codice-esempi/giorno-1-parte-2
```

Ogni sottocartella è un mini-progetto autonomo con `package.json` e `README.md`:

- `01-express-hello/` primo server Express con `nodemon`
- `02-express-routing/` tutti i metodi HTTP, parametri, query, body
- `03-express-middleware/` logger custom e middleware con argomenti
- `04-express-router/` router modulari e refactor del server

- `05-express-crud-inmem/` CRUD completo di utenti finti

Cosa faremo domani

Il Giorno 2 prende lo scheletro di oggi e lo trasforma in un'applicazione vera:

- Approfondimento middleware: **error handler centralizzato**, middleware con argomenti, gestione errori in `async/await`.
- Persistenza vera con **SQLite** (driver `sqlite3` + wrapper `sqlite` per l'API `async/await`). Sostituiamo l'array in memoria con una tabella vera, senza cambiare le route.
- **Repository pattern** formalizzato: tutte le query SQL isolate in un solo file (`userRepository.js`).
- **Validazione input** con `express-validator` e gestione segreti via `dotenv` (`.env`).
- Prima implementazione end-to-end del progetto finale: `POST /api/auth/register` con **hashing password tramite `bcrypt`** e scrittura in SQLite.

Per approfondire

- **Documentazione ufficiale Express:** expressjs.com, soprattutto le sezioni "Getting started" e "Guide".
- **MDN HTTP:** developer.mozilla.org per riferimento su metodi, status code e header.
- **npm:** npmjs.com per esplorare i middleware popolari (`morgan`, `cors`, `helmet`, `compression`).
- **Refactoring UI / API design:** l'articolo "REST API Design Best Practices" su Stack Overflow Blog è un'ottima lettura serale.

Un consiglio prima di andare

Stasera, prima di chiudere il PC, prova a riscrivere il CRUD del capitolo 6 **senza guardare la dispensa**. Quando ti blocchi, riapri solo il pezzo che ti manca e ricomincia. È il modo più veloce per arrivare a domani con i concetti già fissati.